SOFTWARE REQUIREMENTS SPECIFICATION

prepared in conformance with the IEEE recommended practice for software requirements specifications

kvstore

A Distributed, Fault Tolerant, Transactional Key Value Store with an Integrated Operational Intelligence Layer

Document Type:	Software Requirements Specification (SRS)
Project Title:	kvstore: A Distributed Transactional Key Value Store
Document Version:	1.0
Document Status:	Final, Approved for Submission
Prepared For:	Software Engineering Project Evaluation

Submitted By:	Neha Nupur
Class / Section:	MCA II Year
Roll Number:	20251651064

Master of Computer Applications Programme
Department of Computer Applications
Academic Session 2026

Document Control Information

Field	Detail
Document Title	Software Requirements Specification for kvstore
Author	Neha Nupur
Class and Section	MCA II Year
Roll Number	20251651064
Version	1.0
Classification	Academic, Software Engineering Project Deliverable
Applicable Standard	IEEE recommended practice for software requirements specifications (informally referenced throughout as IEEE 830)
Review Status	Reviewed against the implemented source code base of the kvstore project prior to finalisation
Distribution	Submitted for academic evaluation

Revision History

Version	Description	Nature of Change
0.1	Initial draft	Structural outline of all fifteen internal packages and three command line programs established from the source repository.
0.5	Expanded draft	Functional, non functional, and interface requirements written out in full for every subsystem, including the operational intelligence layer.
1.0	Final version	Verification strategy, defect history, traceability matrix, and appendices completed; document reviewed for internal consistency and submitted as final.

Contents

Document Control Information **1**

1 Introduction **4**
1.1 Purpose of this Document . . . 4
1.2 Document Conventions . . . 4
1.3 Intended Audience and Reading Suggestions . . . 4
1.4 Project Scope . . . 5
1.5 Definitions, Acronyms, and Abbreviations . . . 5
1.6 References . . . 8

2 Overall Description **8**
2.1 Product Perspective . . . 8
2.2 Product Functions . . . 9
2.3 User Classes and Characteristics . . . 10
2.4 Operating Environment . . . 10
2.5 Design and Implementation Constraints . . . 10
2.6 Assumptions and Dependencies . . . 11

3 System Architecture and Design Overview **11**
3.1 Architectural Overview . . . 11
3.2 Component Description . . . 11
3.3 Command Line Programs . . . 15
3.4 Deployment Architecture . . . 15

4 Functional Requirements **15**
4.1 Hybrid Logical Clock Requirements . . . 15
4.2 Write Ahead Log Requirements . . . 16
4.3 Raft Consensus Requirements . . . 16
4.4 Multi Version Concurrency Control and Transaction Requirements . . . 17
4.5 Storage Engine Requirements . . . 18
4.6 Network Transport Requirements . . . 18
4.7 Client Library Requirements . . . 19
4.8 Observability and Metrics Requirements . . . 19
4.9 Operational Intelligence Requirements . . . 19
4.10 Transport Layer Security Requirements . . . 21
4.11 Hypertext Transfer Protocol API and Dashboard Requirements . . . 21
4.12 Sharding and Cross Shard Transaction Requirements . . . 21
4.13 Command Line Program Requirements . . . 22

5 External Interface Requirements **22**
5.1 User Interfaces . . . 22
5.2 Hardware Interfaces . . . 23
5.3 Software Interfaces . . . 23
5.4 Communication Interfaces . . . 23

6 Non Functional Requirements **24**
6.1 Performance Requirements . . . 24
6.2 Safety and Reliability Requirements . . . 25
6.3 Availability Requirements . . . 25
6.4 Security Requirements . . . 25
6.5 Maintainability Requirements . . . 25
6.6 Testability Requirements . . . 26
6.7 Portability Requirements . . . 26
6.8 Usability Requirements . . . 26

7 System Models **26**
7.1 Raft Node State Model . . . 26
7.2 Transaction Commit Sequence Model . . . 27
7.3 Data Flow Overview . . . 27

8 Data Requirements **27**
8.1 Logical Data Model . . . 27
8.2 Persistent Data . . . 29
8.3 Data Integrity Requirements . . . 29

9 Verification, Validation, and Testing Strategy **29**
9.1 Testing Philosophy . . . 29
9.2 Fault Injection Testing Harness . . . 29
9.3 Verified Properties . . . 30
9.4 Test Suite Organisation . . . 30
9.5 Acceptance Criteria . . . 31

10 Defect History and Known Limitations **31**
10.1 Notable Defects Found and Corrected During Development . . . 31
10.2 Known Limitations . . . 32

11 Appendices **32**
11.1 Appendix A: Requirement Traceability Overview . . . 32
11.2 Appendix B: Command Line Flag Summary . . . 33
11.3 Appendix C: Glossary Cross Reference . . . 33

1 Introduction

1.1 Purpose of this Document

This document is the Software Requirements Specification, referred to throughout as the SRS, for the software system named kvstore. The purpose of this document is to describe, in a complete, unambiguous, verifiable, and formally structured manner, every functional requirement, non functional requirement, interface requirement, and design constraint that governs the kvstore system as it has been designed, implemented, and tested. The document is written in conformance with the spirit of the IEEE recommended practice for software requirements specifications, adapted to the scale and academic context of a Master of Computer Applications software engineering project, and it is intended to serve simultaneously as a specification of intent, a record of what has actually been built, and a reference against which the correctness of the delivered system can be independently verified.

A software requirements specification exists to remove ambiguity from the relationship between what a system is expected to do and what it actually does. In the case of kvstore, this relationship is unusually well defined because the system was built iteratively, tested continuously through a dedicated fault injection harness, and audited for consistency between its own design documentation and its own source code prior to the preparation of this specification. Every requirement recorded in this document is therefore traceable either to an implemented and tested capability of the system or, where explicitly noted, to a documented and deliberate limitation of the current implementation.

1.2 Document Conventions

Throughout this document, the auxiliary verb shall is used exclusively to denote a mandatory requirement that the implemented system satisfies or must satisfy. The auxiliary verb should is reserved for a recommended but non mandatory property. The auxiliary verb may is used to denote an optional capability that is present in the system but not required for correct operation in every configuration. Every functional requirement is assigned a unique identifier of the form SRS FR followed by a three digit number, every non functional requirement is assigned an identifier of the form SRS NFR followed by a three digit number, and every interface requirement is assigned an identifier of the form SRS IR followed by a three digit number, so that each requirement can be referenced unambiguously from the traceability matrix presented in the appendices. Package names, file names, function names, command line flags, and Hypertext Transfer Protocol routes are set in a fixed width typeface to distinguish them clearly from ordinary prose. This document intentionally avoids the hyphen character in ordinary prose in order to satisfy an explicit formatting requirement placed on its preparation; compound technical terms that are conventionally hyphenated in general software engineering literature, such as key value store, write ahead log, multi version concurrency control, and cross shard transaction, are therefore rendered here as open compounds of two or more words rather than as hyphenated compounds, without any change in their technical meaning.

1.3 Intended Audience and Reading Suggestions

This document is intended for four distinct classes of reader. The first is an academic evaluator or examiner assessing the software engineering project against the customary expectations of requirements analysis, system design, and verification planning; such a reader is advised to read Sections 1 through 3 for context, Sections 4 through 6 for the substantive requirement statements, and Section 9 for confirmation that the requirements have been tested rather than merely asserted. The

second is a future maintainer or extender of the kvstore codebase, who is advised to read Section 3 for the architectural overview and Section 4 for the module by module functional requirements before making any change to the source code. The third is a quality assurance reviewer, who is advised to concentrate on Section 6, which states the non functional requirements, and Section 9, which describes the verification strategy and the fault injection testing harness used to validate correctness under adversarial conditions. The fourth is any reader encountering the kvstore project for the first time with no prior background in distributed systems; such a reader is advised to read the document from the beginning, since Section 2 and Section 3 build up the necessary conceptual vocabulary, including consensus, replication, and snapshot isolation, before that vocabulary is used in the formal requirement statements of later sections.

1.4 Project Scope

kvstore is a distributed, replicated, transactional key value store implemented entirely in the Go programming language, using exclusively the Go standard library, with no third party software dependency of any kind at any layer of the system. A key value store is a database whose data model consists of a collection of string keys, each of which is associated with an arbitrary byte sequence value; the operations exposed to a client are limited by design to writing a value under a key, reading the value currently stored under a key, deleting a key, and combining several such operations into one atomic, all or nothing transaction spanning multiple keys.

The scope of the kvstore project extends well beyond this simple data model to encompass everything required to make that data model survive and remain useful in a realistic, imperfect operating environment in which individual machines fail, networks partition, messages are delayed or lost, and operators need visibility into what the running system is doing. Within scope, therefore, are: a Raft based consensus layer providing fault tolerant replication across multiple nodes; a multi version concurrency control layer providing snapshot isolated transactions with deterministic, replica consistent conflict detection; a checksummed, crash safe write ahead log providing durability; a hybrid logical clock providing a globally consistent event ordering; an optional Transport Layer Security encrypted network layer, including an optional mutual authentication mode; a dependency free observability and metrics layer; an operational intelligence layer built from transparent statistical techniques, including anomaly detection, multivariate correlation, trend forecasting, a leadership and load advisor, an adaptive automatic tuner, and a natural language query interface with an optional large language model integration for query fallback and root cause diagnosis; a Hypertext Transfer Protocol application programming interface and an embedded browser based live dashboard; a horizontal sharding layer using rendezvous hashing to route requests across multiple independent clusters; a two phase commit coordinator providing atomic, crash recoverable transactions that span more than one shard; and three command line programs, namely the cluster node server, a command line client, and the sharding gateway router, that assemble all of the above into runnable software.

Explicitly out of scope, by deliberate design decision rather than by omission, are: a fully general distributed transaction system supporting arbitrary reads that span multiple independent shards; any trained or learned machine learning model of any kind within the operational intelligence layer, which is built exclusively from classical, auditable statistical methods; and any fully automatic, non optional leadership transfer behaviour that is not the result of an explicit operator configuration choice.

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
Raft	A consensus algorithm by which a group of computers agrees on a single, ordered sequence of operations despite crashes, restarts, and unreliable message delivery, designed for comprehensibility relative to earlier consensus algorithms.
Consensus	The general problem of a distributed set of processes agreeing on a single value or ordered sequence of values even when some processes may fail.
MVCC	Multi Version Concurrency Control, a technique for supporting concurrent transactions by retaining multiple timestamped versions of each data item rather than a single current value.
Snapshot Isolation	A transaction isolation level under which a transaction observes a consistent snapshot of the entire database as it existed at the transaction's start time, unaffected by concurrent transactions that commit later.
HLC	Hybrid Logical Clock, a timestamping mechanism combining a physical wall clock reading with a logical counter and a node identifier so as to provide a globally unique, causally consistent ordering of events.
WAL	Write Ahead Log, a durability mechanism in which a description of a change is appended to a log file and flushed to physical storage before the change is considered permanent.
Fsync	An operating system call that forces buffered writes to a file to be physically flushed to the underlying storage device.
Quorum	A majority subset of the members of a cluster whose agreement is required before an operation can be considered committed.
Split Brain	A failure scenario in which two disjoint subsets of a cluster each incorrectly believe themselves to hold a valid majority and independently accept conflicting writes.
Joint Consensus	A two step technique for safely changing cluster membership in a Raft based system by transitioning through an intermediate configuration that requires agreement from majorities of both the old and the new membership lists.
First Committer Wins	A conflict resolution rule under which, of two concurrently executing transactions that both attempt to modify the same key, only the one whose commit is recorded first is permitted to succeed, the other being aborted.

Term	Definition
Intent	A temporary, exclusive lock placed on a key on behalf of a transaction that has been tentatively prepared but not yet finally resolved as committed or aborted.
Two Phase Commit	A distributed transaction protocol in which a coordinator first asks every participant to prepare a transaction and only proceeds to instruct a commit once every participant has agreed to prepare, thereby achieving atomicity across independent systems.
Rendezvous Hashing	A hashing based technique, also known as highest random weight hashing, for assigning a key to one of several destinations such that the addition or removal of a single destination remaps only the minimal necessary fraction of keys.
TLS	Transport Layer Security, a cryptographic protocol providing confidentiality and integrity for network communication.
Mutual TLS	A mode of Transport Layer Security in which both the client and the server are required to present and validate a cryptographic certificate, rather than only the server.
EWMA	Exponentially Weighted Moving Average, a smoothing technique in which more recent observations are given greater weight than older observations.
Welford's Algorithm	A numerically stable, single pass algorithm for computing running mean and variance of a stream of numbers.
Pearson Correlation Coefficient	A statistical measure, ranging from negative one to positive one, of the linear relationship between two sets of paired numerical observations.
Least Squares Regression	A standard statistical method for fitting a straight line through a set of data points by minimising the sum of the squared vertical distances between the line and the points.
Coefficient of Determination, R Squared	A statistic reporting the proportion of variance in a dependent variable that is explained by a fitted regression line, used here as a confidence measure for trend forecasts.
SSE	Server Sent Events, a web standard allowing a server to push a continuous stream of events to a connected browser over a single, long lived Hypertext Transfer Protocol connection.
LLM	Large Language Model, a class of machine learning model used in this project exclusively as an optional, narrowly constrained, and non authoritative auxiliary component.

Term	Definition
API	Application Programming Interface.
CLI	Command Line Interface.
REST	Representational State Transfer, an architectural style for networked application interfaces built around resources addressed by uniform resource locators and manipulated through standard Hypertext Transfer Protocol methods.

1.6 References

1. IEEE Recommended Practice for Software Requirements Specifications, IEEE Standard 830, Institute of Electrical and Electronics Engineers.
2. Ongaro, Diego, and Ousterhout, John. In Search of an Understandable Consensus Algorithm, the original Raft paper, together with Ongaro's associated doctoral dissertation, which is the source of the joint consensus membership change technique implemented in this project.
3. Berenson, Hal, et al. A Critique of ANSI SQL Isolation Levels, the source of the formal definition of snapshot isolation used as the transaction isolation guarantee provided by this project's multi version concurrency control layer.
4. Thaler, David, and Ravishankar, Chinya. A Name Based Mapping Scheme for Rendezvous, the source of the rendezvous hashing technique, also called highest random weight hashing, used by the sharding router.
5. Gray, Jim, and Lamport, Leslie. Consensus on Transaction Commit, a standard reference on the two phase commit protocol underlying the cross shard transaction coordinator.
6. The project's own internal documentation, comprising README.md, PLAN.md, and PROJECT_OVERVIEW.md, contained within the submitted source repository, which together form the primary source material from which this specification has been derived.
7. The Go Programming Language standard library documentation, the sole software dependency of the implemented system.

2 Overall Description

2.1 Product Perspective

kvstore is a new, self contained, ground up system rather than an extension of an existing product. It occupies the same general category of software as widely used industrial distributed databases and coordination systems, but it was built specifically as an educational and demonstrative software engineering artefact, with an explicit and unusual constraint that no third party library of any kind, whether for networking, consensus, storage, or serialisation, is used anywhere in the system. This constraint was adopted deliberately so that every layer of the system's behaviour could be fully

understood, tested, and reasoned about by inspecting this codebase alone. The system is composed of three independently runnable programs, a server program that forms one member of a replicated cluster, a command line client for manual and scripted interaction, and a sharding gateway that routes requests across multiple independent clusters, all three of which are built from a shared set of internal packages arranged in a strict, one directional dependency hierarchy in which lower level packages never depend upon higher level ones.

2.2 Product Functions

At the highest level of abstraction, the kvstore system performs the following principal functions.

- It stores and retrieves arbitrary byte sequence values addressed by string keys, replicating every write across a configurable number of cluster members using the Raft consensus algorithm so that the failure of a minority of nodes never results in data loss or a period of complete unavailability.
- It executes multi key transactions with snapshot isolation and deterministic, replica consistent conflict detection, so that a client may read and write several keys as a single atomic unit with a well defined guarantee about what changes made by other, concurrently executing transactions are and are not visible.
- It guarantees durability of every acknowledged write through a checksummed, forcibly flushed write ahead log, such that no operation the system has reported as successful can be lost by a subsequent crash of the process or the underlying operating system.
- It supports safe, live changes to cluster membership, allowing nodes to be added to or removed from a running cluster without any window of vulnerability to a simultaneous, conflicting dual leadership.
- It exposes rich, continuously updated observability data, including request latency, throughput, and error rate, in both an industry standard text exposition format and a browser consumable format.
- It continuously analyses its own observability data using transparent statistical techniques to detect anomalies, correlate related signals, forecast trends, recommend leadership transfers to healthier nodes, and automatically tune internal operational parameters, optionally augmented by a narrowly constrained large language model integration for natural language querying and root cause explanation.
- It exposes a full featured Hypertext Transfer Protocol application programming interface and an embedded, dependency free, browser based live dashboard, allowing both human operators and other software to interact with the cluster without needing to implement the system's internal binary network protocol.
- It optionally encrypts all network traffic between cluster members and between clients and the cluster using Transport Layer Security, with an optional mutual authentication mode for environments requiring client identity verification.
- It scales horizontally beyond the capacity of a single Raft cluster by routing keys across multiple independent clusters using rendezvous hashing, and it supports atomic, crash recoverable, multi key write transactions that span more than one such cluster using a dedicated two phase commit coordinator.

2.3 User Classes and Characteristics

Four classes of user interact with the kvstore system, each with distinct needs and expectations.

Application developers interact with the system programmatically, either through the internal client library, through the command line client, or through the Hypertext Transfer Protocol application programming interface, and require a simple, predictable, well documented interface for storing, retrieving, and transactionally updating data without needing to understand the internal consensus mechanism.

Cluster operators are responsible for deploying, configuring, monitoring, and maintaining a running kvstore cluster. This class of user requires clear visibility into cluster health, meaningful and actionable alerts when something is wrong, and safe, well understood procedures for routine maintenance activities such as adding a node, removing a node, or transferring leadership away from a node that needs to be taken down.

System integrators are responsible for embedding kvstore into a larger software system, and require stable, well specified network protocols and application programming interfaces, including the sharding and cross shard transaction interfaces, so that kvstore can be composed reliably with other software components.

Project evaluators and maintainers, the class most directly relevant to the academic context of this document, require a complete, honest, and precisely specified description of what the system does, how it is structured, and, critically, what its known limitations are, so that the quality of the engineering can be assessed on its actual, demonstrated merits rather than on unverified claims.

2.4 Operating Environment

The system is implemented in the Go programming language and is designed to run on any operating system and processor architecture supported by the standard Go compiler toolchain, including the commonly used Linux, macOS, and Windows operating systems on both established and modern processor architectures. Each cluster member requires local persistent storage for its write ahead log, snapshot files, and hard state file, organised into a per node data directory. The system requires only ordinary Transmission Control Protocol and Internet Protocol network connectivity between cluster members and between clients and the cluster; no specialised network hardware or protocol is required. The optional containerised deployment configuration, provided through a Docker Compose specification, demonstrates a realistic topology of two independent three node Raft clusters together with a sharding gateway, and requires only a standard container runtime.

2.5 Design and Implementation Constraints

The overriding design constraint placed on this project, and honoured without exception across every internal package and command line program, is that the system uses no third party software library whatsoever; every capability, from binary serialisation to statistical computation to cryptographic certificate generation, is built using only the Go standard library. A second constraint is architectural: the internal packages are arranged in a strict, acyclic dependency hierarchy, so that the operational intelligence layer, for example, can be compiled, tested, and reasoned about without any dependency on the consensus, storage, or network code, and so that the lowest level packages, namely the hybrid logical clock and the write ahead log, have no dependency on anything above them in the hierarchy. A third constraint concerns testing philosophy: the project's own design documentation states an explicit

preference for testing new functionality against real infrastructure, meaning real network sockets, real Transport Layer Security handshakes, and real multiple node clusters, rather than against simplified test doubles, wherever that is practical, precisely because a simplified stand in can too easily be constructed in a way that merely confirms an author's existing assumption rather than exposing a genuine defect.

2.6 Assumptions and Dependencies

The specification of this system assumes that the underlying operating system's file system provides the ordinary guarantees expected of a Portable Operating System Interface compliant file system, specifically that a successful fsync call durably persists previously written bytes to physical storage. It assumes that the clocks of different physical machines participating in a cluster remain within a bounded drift of one another, nominally within the configurable maximum clock skew tolerance, since the correctness properties the system relies upon come from the node identifier component embedded in every hybrid logical clock timestamp rather than from active, wire level clock synchronisation between nodes, a point discussed in full in Section 3 and Section 10 of this document. It assumes that network communication between nodes, while it may be delayed, dropped, duplicated, or reordered, is never maliciously altered in transit unless Transport Layer Security is explicitly enabled. It assumes the availability of a standard Go compiler toolchain for building the system from source, and, only for the strictly optional large language model integrations within the operational intelligence layer, the availability of network connectivity to an external large language model provider and a valid credential supplied by the operator through an environment variable; the system is fully functional in every other respect with this optional dependency entirely absent.

3 System Architecture and Design Overview

3.1 Architectural Overview

The kvstore system is organised as a strict, layered hierarchy of internal Go packages, in which every package occupies a specific horizontal layer and is permitted to depend only on packages in layers strictly below its own. This design decision is what allows the lowest layers to be tested in total isolation and what allows the operational intelligence layer, despite sitting conceptually close to the top of the system, to be tested with no cluster of any kind running. Figure 1 presents this layered dependency structure graphically, reading from the most fundamental, dependency free layer at the bottom to the fully integrated, externally facing layer at the top; each shaded band on the left marks one dependency layer, and every arrow points from a dependent package down to the package it depends upon.

3.2 Component Description

3.2.1 Hybrid Logical Clock, internal/hlc

This package produces globally comparable timestamps by combining a physical wall clock reading in nanoseconds, a logical counter that advances whenever two events would otherwise collide on the identical physical reading, and the identifier of the node that produced the timestamp, which guarantees uniqueness even under a physical clock collision. Timestamps are compared by physical component first, logical component second, and node identifier third, producing a strict total ordering across the entire cluster. The package exposes a method to advance the local clock and a separate method to fold in a timestamp observed from a remote node, together with a configurable maximum tolerated drift

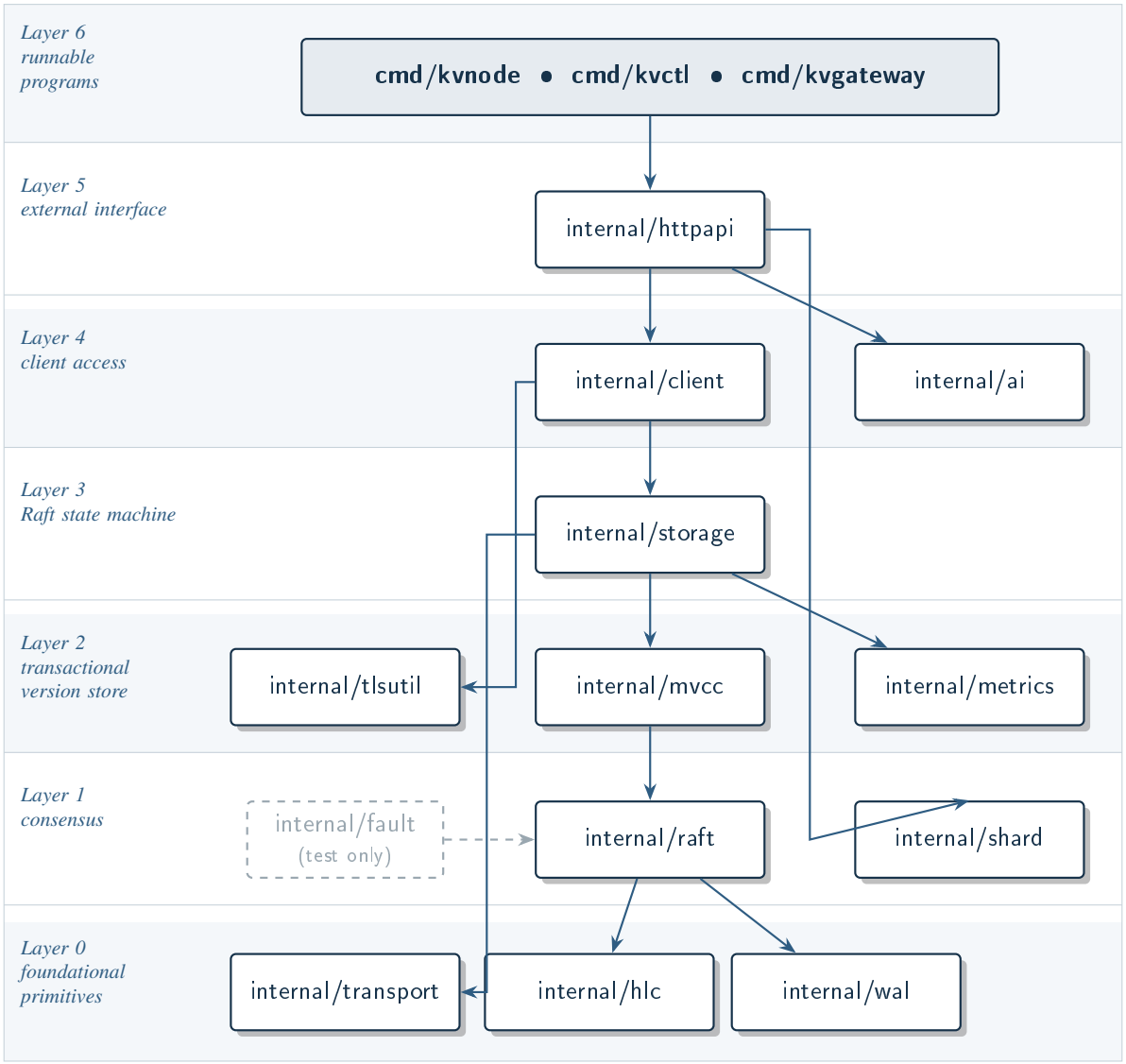


Figure 1: Layered package dependency structure of the kvstore system. Arrows point from a dependent package to the package it depends upon; shaded bands mark the seven dependency layers described in Section 3.2. The internal/ai package depends only on two narrow interfaces supplied by internal/httpapi and has no direct dependency on consensus, storage, or transport code.

and a hook for injecting an artificial clock offset for testing purposes.

3.2.2 Write Ahead Log, internal/wal

This package provides a generic, content agnostic durability primitive. It accepts arbitrary byte sequence records, appends each one to a log file together with a checksum, and forces a physical flush of that write to the storage device before considering the record durable. During replay after a restart, any record whose checksum fails to match, or any record that is truncated, is treated as the effective end of the log, so that the specific kind of corruption a crash occurring mid write can produce is safely tolerated. The Raft consensus package is the sole consumer of this mechanism, using it to persist log entries, the current term and vote record, and periodic snapshots.

3.2.3 Raft Consensus, internal/raft

This is the largest and most intricate package in the system, implementing leader election with randomised election timeouts, log replication with a fast conflict back off optimisation for catching up lagging followers, log compaction through periodic snapshotting with a dedicated snapshot transfer mechanism for followers that have fallen too far behind to be caught up incrementally, and safe, live cluster membership changes using the joint consensus technique described in the original Raft dissertation. Every durable piece of Raft state, namely the replicated log, the current term and vote, the cluster configuration, and periodic snapshots, is written through the write ahead log package described above.

3.2.4 Multi Version Concurrency Control, internal/mvcc

This package stores, for every key, a chronologically ordered chain of versions rather than a single current value, each version tagged with the hybrid logical clock timestamp at which it was written. It provides snapshot isolated transactional reads, in which a transaction observes precisely the versions that were the most recently committed as of its own start timestamp, and it provides first committer wins conflict detection at the moment a transaction's write set is deterministically applied from the Raft log, ensuring that every replica independently reaches an identical accept or abort decision for every transaction. It additionally provides a garbage collection mechanism that reclaims versions no longer visible to any still active transaction, and an intent based locking mechanism used by the cross shard transaction coordinator described later in this section.

3.2.5 Storage Engine, internal/storage

This package is the Raft state machine: the specific component to which Raft hands every committed log entry, in order, on every node, for interpretation and application to the actual key value data. It wires the hybrid logical clock, the write ahead log through Raft, and the multi version concurrency control version store into one coherent transactional application programming interface exposing begin, read, write, delete, and commit operations to the rest of the system.

3.2.6 Network Transport, internal/transport

This package implements a minimal, purpose built, connection per call network protocol, in which a single structured envelope message is encoded using Go's built in binary serialisation format, sent over a freshly opened Transmission Control Protocol connection, and answered with a single reply before the connection is closed. The same server dispatches both internal Raft protocol messages between cluster members and external client key value requests, and it returns a specific reply, carrying a hint identifying the current leader when known, whenever a client request arrives at a node that is not

currently the leader. Transport Layer Security encryption is supported as an entirely optional, additive layer on top of this protocol.

3.2.7 Client Library, internal/client

This package provides the single, shared implementation of the logic every client of the cluster needs: discovering which node is currently the leader, automatically redirecting a request when it is sent to a node that is not the leader, and retrying sensibly across multiple candidate addresses when some nodes are temporarily unreachable.

3.2.8 Metrics and Observability, internal/metrics

This package provides a dependency free instrumentation library offering counters for monotonically increasing totals, gauges for values that move up and down, and histograms that compute count, sum, minimum, maximum, a numerically stable running mean and standard deviation using Welford's algorithm, an exponentially weighted moving average for recent behaviour, and approximate percentile estimates using logarithmically sized buckets. Every request handled by the network server is automatically measured, and the resulting data is exposed both in the standard Prometheus text exposition format and in a plain JSON format for the embedded dashboard.

3.2.9 Operational Intelligence, internal/ai

This package implements the operational intelligence layer described at length in Section 4.9 of this document. It is deliberately isolated from every other layer of the system, depending on no consensus, storage, or network code whatsoever; the data it needs is supplied to it exclusively through two narrowly defined interfaces, a statistics provider interface and an executor interface, which some other component of the system must implement and supply. This isolation is what allows the entire layer to be unit tested completely on its own.

3.2.10 Transport Layer Security Utilities, internal/tlsutil

This package provides helper functions for generating a temporary, self signed certificate suitable for local development, and for loading a real certificate and private key pair for production deployment, including support for mutual Transport Layer Security in which both connection endpoints must present a valid certificate.

3.2.11 Hypertext Transfer Protocol API and Dashboard, internal/httpapi

This package is the component permitted to depend on every other package in the system, because its specific responsibility is to wire all of them together and expose the resulting functionality externally through a REST style application programming interface, a live Server Sent Events stream, and an embedded, self contained browser dashboard. It also contains the adapter implementation of the two interfaces the operational intelligence layer requires, and it implements cross node health polling that allows the leadership advisor to compare every node in the cluster rather than only the node a caller happens to be looking at.

3.2.12 Sharding, internal/shard

This package implements rendezvous hashing, computing, for a given key, a deterministic numeric score against every configured shard using the SHA two hundred fifty six hash function, and routing that key to the shard with the highest score. This technique ensures that changing the number of configured shards remaps only the minimal necessary fraction of existing keys rather than the

overwhelming majority of them.

3.2.13 Fault Injection Harness, internal/fault

This package, used exclusively by the test suite and never included in a production deployment, provides an in process simulated network capable of partitioning a cluster into groups that cannot exchange messages, introducing configurable message delay and loss, and killing and restarting simulated nodes in a way that genuinely reopens and replays that node's on disk write ahead log, exercising the identical code path a real process crash and restart would exercise.

3.3 Command Line Programs

Three command line programs assemble the packages described above into software that a person or another program actually runs. `cmd/kvnode` is the server program; one running instance of it is one member of exactly one Raft cluster, and it is configured through an extensive set of command line flags governing its network addresses, data directory, consensus timing parameters, optional Transport Layer Security settings, and optional operational intelligence settings. `cmd/kvctl` is a small command line client, built directly on the shared client library, supporting the basic operations of reading, writing, deleting, and checking node status, intended for manual interaction and for use within scripts. `cmd/kvgateway` is the sharding router program, configured with the network addresses of every independent shard cluster it should route requests to, and, when supplied with its own data directory, additionally supporting the atomic cross shard transaction endpoint described in Section 4.11.

3.4 Deployment Architecture

A representative deployment, demonstrated through an included Docker Compose specification, consists of two independent, three node Raft clusters, functioning as two shards, together with a single sharding gateway container configured with a durable data directory of its own so that its two phase commit decision log survives a container restart. Each of the six cluster member containers maintains its own dedicated, named, persistent storage volume for its data directory. Every cluster member and the gateway may optionally expose the embedded Hypertext Transfer Protocol dashboard, and setting a single environment variable before deployment activates the optional large language model fallback for the natural language query interface on every node simultaneously, with no other change to the deployment required.

4 Functional Requirements

This section states, module by module, the complete set of functional requirements the kvstore system satisfies. Each requirement is stated using the mandatory auxiliary verb shall and is assigned a unique identifier for traceability purposes, as explained in Section 1.2.

4.1 Hybrid Logical Clock Requirements

SRS FR 001 The system shall generate, for every event requiring a timestamp, a composite value consisting of a physical wall clock reading, a logical counter, and the identifier of the generating node.

SRS FR 002 The system shall increase the logical counter component of a generated timestamp whenever the physical wall clock reading has not advanced relative to the previous timestamp generated by the same node.

SRS FR 003 The system shall provide a comparison function over generated timestamps that orders first by physical component, second by logical component, and third by node identifier, yielding a strict total order across the entire cluster.

SRS FR 004 The system shall provide a mechanism for folding a timestamp received from a remote node into the local clock state such that causal ordering between the local node and the remote node is preserved when that mechanism is invoked.

SRS FR 005 The system shall support a configurable maximum tolerated clock drift and shall support the artificial injection of a clock offset for the purpose of automated testing of clock skew handling.

SRS FR 006 The system shall document, as a known and explicit limitation rather than as an implicit assumption, that no message currently transmitted over the network protocol carries a remote timestamp for automatic folding during ordinary production operation, and that cross node ordering during ordinary operation instead depends on the node identifier component of every timestamp together with each node's physical clock remaining within the configured maximum tolerated drift of its peers.

4.2 Write Ahead Log Requirements

SRS FR 007 The system shall append an arbitrary byte sequence record to a log file together with a checksum computed over the contents of that record.

SRS FR 008 The system shall force a physical flush of every appended record to the underlying storage device before reporting that record as durably persisted.

SRS FR 009 The system shall, during replay of a log file after a restart, recompute the checksum of every stored record and shall treat the first record whose recomputed checksum does not match its stored checksum, or the first record that is truncated, as the effective end of the log, discarding nothing that precedes it and trusting nothing beyond it.

SRS FR 010 The system shall use this write ahead log mechanism to persist every Raft log entry, the current term and vote record, and every periodic snapshot for each cluster node.

4.3 Raft Consensus Requirements

SRS FR 011 The system shall maintain, for every cluster node at every moment, exactly one of the three roles leader, follower, or candidate.

SRS FR 012 The system shall elect at most one leader per Raft term across the entire cluster, verified specifically under contested, near simultaneous election conditions.

SRS FR 013 The system shall use a randomised election timeout, configurable through a minimum and maximum bound, so that followers do not simultaneously become candidates.

SRS FR 014 The system shall require that a candidate's replicated log be at least as up to date as a voting node's own log before that voting node grants its vote, and shall permit a voting node to grant at most one vote per term.

SRS FR 015 The system shall replicate every operation accepted by the leader to every follower and shall consider that operation committed only once a majority of the cluster, including the leader itself, has durably stored it.

SRS FR 016 The system shall detect the point at which a follower's log diverges from the leader's log and shall bring that follower back into agreement using an accelerated conflict back off mechanism rather than retransmitting one entry at a time.

SRS FR 017 The system shall periodically compact its log through snapshotting and shall discard log entries made redundant by a completed snapshot.

SRS FR 018 The system shall transfer a complete snapshot directly to any follower that has fallen so far behind that the leader has already discarded the log entries that follower requires, after which that follower shall resume ordinary incremental replication.

SRS FR 019 The system shall support live changes to cluster membership, including the addition and removal of voting members, using a joint consensus configuration that requires majority agreement from both the old and the new membership lists during the transitional period.

SRS FR 020 The system shall automatically propose the finalising configuration entry, dropping the old membership list, once the joint configuration entry has itself been committed.

SRS FR 021 The system shall support a deliberate, voluntary leadership transfer from the current leader to a specified follower, ensuring the target follower's log is fully caught up before instructing it to begin an election immediately, without waiting for its normal election timeout.

SRS FR 022 The system shall never permit a network partition to allow a minority subset of the cluster to elect a leader or to commit a write, ensuring that a full network partition can never result in the two disjoint sides of the cluster independently and silently diverging into inconsistent data.

4.4 Multi Version Concurrency Control and Transaction Requirements

SRS FR 023 The system shall store, for every key, an ordered chain of versions, each version tagged with the hybrid logical clock timestamp at which it was written.

SRS FR 024 The system shall assign every new transaction a beginning timestamp representing the instant the transaction started.

SRS FR 025 The system shall ensure that every read performed by a transaction, for the remainder of that transaction's lifetime, returns the version of the requested key that was most recently committed as of that transaction's beginning timestamp, regardless of any transaction that commits later while this transaction is still active.

SRS FR 026 The system shall reject, at commit time, any transaction for which some key it wrote has had a newer version committed since that transaction's beginning timestamp, applying a first committer wins conflict resolution rule.

SRS FR 027 The system shall perform the authoritative conflict check for a committing transaction deterministically, at the moment that transaction's write set is applied from the Raft log on every replica, so that every replica independently reaches the identical commit or abort decision.

SRS FR 028 The system shall additionally perform an optimistic, non authoritative conflict check before a transaction is proposed through Raft at all, so that a transaction that is already certain to fail can be rejected quickly without incurring the cost of a full consensus round trip.

SRS FR 029 The system shall assign every transaction a unique identifier derived deterministically from that transaction's hybrid logical clock beginning timestamp tuple, ensuring uniqueness across the whole cluster and across process restarts.

SRS FR 030 The system shall reclaim, through a garbage collection process, any version of a key older than the beginning timestamp of the oldest still active transaction in the cluster, while never reclaiming the single most recent remaining version of any key.

SRS FR 031 The system shall support a temporary, exclusive intent lock on a set of keys on behalf of a transaction that has been tentatively prepared but not yet resolved, during which no ordinary read is permitted to observe the locked keys in an uncommitted state.

4.5 Storage Engine Requirements

SRS FR 032 The system shall expose a transactional application programming interface supporting the operations begin, get, put, delete, and commit, callable by code elsewhere in the system.

SRS FR 033 The system shall, upon a commit request, propose the transaction's complete write set through Raft as a single log entry and shall wait for that entry to be safely and deterministically applied to a majority of the cluster before reporting success to the caller.

SRS FR 034 The system shall function as the Raft state machine, meaning it shall receive every committed log entry from Raft in order, on every node, and shall interpret and apply that entry to the actual key value data.

4.6 Network Transport Requirements

SRS FR 035 The system shall encode every request as a single structured envelope message using a built in binary serialisation format and shall transmit it over a freshly opened connection for each individual request.

SRS FR 036 The system shall dispatch both internal Raft protocol messages and external client key value requests through the same server process and the same listening network address.

SRS FR 037 The system shall, when a client request arrives at a node that is not the current leader, respond with a specific reply indicating that condition, and shall include, whenever known, the network address of the current leader as a hint.

SRS FR 038 The system shall support an entirely optional Transport Layer Security encrypted mode for every connection, activated additively without changing default behaviour when disabled.

SRS FR 039 The system shall support an optional mutual Transport Layer Security mode requiring every connecting client to present a certificate signed by a configured trusted authority.

4.7 Client Library Requirements

SRS FR 040 The system shall provide a client library capable of discovering which cluster node is currently the leader.

SRS FR 041 The system shall automatically redirect a client request to the correct leader node when an initial request is answered with a not leader reply carrying a leader hint.

SRS FR 042 The system shall retry a client request across multiple candidate cluster addresses when one or more addresses are temporarily unreachable, rather than failing immediately upon the first unreachable address.

4.8 Observability and Metrics Requirements

SRS FR 043 The system shall provide monotonically increasing counters, freely adjustable gauges, and histograms as its three fundamental instrumentation primitives.

SRS FR 044 The system shall compute, for every histogram, the count, sum, minimum, maximum, running mean, and running standard deviation of recorded observations using a numerically stable, single pass algorithm.

SRS FR 045 The system shall compute an exponentially weighted moving average for every histogram, emphasising the most recent observations over older ones.

SRS FR 046 The system shall compute approximate percentile estimates for every histogram using a fixed set of logarithmically sized buckets.

SRS FR 047 The system shall automatically measure the latency and success or failure outcome of every client request handled by the network server, broken down by the specific operation performed.

SRS FR 048 The system shall expose accumulated metrics data in both the standard Prometheus text exposition format and a plain JSON format.

4.9 Operational Intelligence Requirements

SRS FR 049 The system shall detect anomalies in a monitored metric series using two exponentially weighted moving averages tracked in parallel, one representing a slowly moving historical baseline and one representing rapidly moving current behaviour, and shall flag an anomaly when the current estimate diverges sufficiently from the baseline relative to that metric's own recent variability.

SRS FR 050 The system shall compute the baseline variance estimate used in anomaly detection using a mechanism that is robust against being dragged upward by a sustained anomaly, so that a persistent problem does not gradually retrain the baseline to consider that problem normal.

SRS FR 051 The system shall retain, for every tracked metric, its recent raw observation history in a bounded, fixed size rolling buffer, distinct from that metric's smoothed moving average values.

SRS FR 052 The system shall, upon the detection of a critical anomaly in one metric, automatically compute the Pearson correlation coefficient between that metric and every other tracked metric over the same recent time window, and shall report the resulting correlated series together with the numerical strength of each correlation.

SRS FR 053 The system shall fit a least squares regression line through a tracked metric's recent raw history and shall project that line forward to produce a trend forecast, reporting alongside that forecast a coefficient of determination indicating the goodness of fit of the underlying trend line.

SRS FR 054 The system shall continuously score every node in the cluster on a combination of recently observed latency, error rate, replication lag, and queue depth.

SRS FR 055 The system shall recommend a leadership transfer away from the current leader when a follower is clearly and consistently healthier by the node scoring mechanism, applying a hysteresis margin to prevent the recommendation from oscillating between two nodes of similar health.

SRS FR 056 The system shall never allow the leadership advisor to directly instruct the consensus layer to perform a transfer; the advisor shall only ever produce a recommendation, with any actual transfer requiring either an explicit operator request or an explicit, separately configured automatic transfer setting.

SRS FR 057 The system shall adaptively adjust the garbage collection interval and the automatic snapshotting threshold using a proportional feedback controller driven by the currently observed write rate and the degree of accumulated multi version concurrency control version bloat.

SRS FR 058 The system shall answer common, plainly phrased natural language questions about cluster state, including questions about the number of stored keys, the identity of the current leader, recently detected anomalies, and a forecast for a named metric, using a fully deterministic, rule based parser requiring no external network dependency.

SRS FR 059 The system shall, only when an external large language model credential has been explicitly configured by the operator, fall back to a large language model for natural language questions the deterministic parser does not recognise, and shall constrain that large language model to selecting exclusively from among the same fixed, pre validated set of read only operations the deterministic parser itself supports, independently verifying every selection against that fixed list before executing it.

SRS FR 060 The system shall, only when an external large language model credential has been explicitly configured, optionally hand already computed anomaly and correlation evidence to a large language model to obtain a plain language, prose only root cause hypothesis and suggested next step, and shall ensure this diagnosis capability never returns anything that is executed, runs asynchronously under a rate limit, and never blocks the system's own continuous monitoring loop.

SRS FR 061 The system shall degrade gracefully and silently whenever the optional large language model call fails or is not configured, such that the deterministic anomaly and correlation insights already computed and recorded remain fully valid and available regardless.

SRS FR 062 The system shall implement the entire operational intelligence layer without any direct dependency on the consensus, storage, or network transport code, obtaining all required cluster data exclusively through two narrowly defined interfaces supplied by an external adapter.

4.10 Transport Layer Security Requirements

SRS FR 063 The system shall generate a temporary, self signed certificate suitable for local development use when Transport Layer Security is enabled without an explicitly supplied certificate and key.

SRS FR 064 The system shall load a real certificate and private key pair supplied by the operator for production deployment when explicitly configured to do so.

SRS FR 065 The system shall support requiring every connecting client to present a certificate signed by a configured trusted authority when mutual Transport Layer Security is enabled.

4.11 Hypertext Transfer Protocol API and Dashboard Requirements

SRS FR 066 The system shall expose, on every node for which the Hypertext Transfer Protocol interface is enabled, a REST style application programming interface supporting reading, writing, and deleting a single key, submitting a multi key transaction, browsing the locally visible set of keys, retrieving node status, retrieving metrics in either supported format, and interacting with every capability of the operational intelligence layer.

SRS FR 067 The system shall expose a continuous, live streaming feed of cluster events using the Server Sent Events web standard, chosen specifically because the required data flow is one directional, from server to browser, and because the standard provides automatic client side reconnection without additional implementation effort.

SRS FR 068 The system shall serve a single, self contained, embedded browser dashboard requiring no separately deployed files and no externally fetched resources, presenting a live visual representation of the cluster arranged as nodes in a circle with visible connections to the current leader that pulse on each heartbeat, live charts of latency, throughput, and error rate, a browser for currently stored keys, a feed of operational intelligence insights, and a text input for submitting natural language questions.

SRS FR 069 The system shall allow every node to optionally poll the status endpoint of every other configured peer node over ordinary Hypertext Transfer Protocol, building a complete, honestly self reported picture of cluster wide health for the leadership advisor, and shall ensure that a node with no peer polling configured limits itself to what it can observe locally and never recommends a leadership transfer in that configuration.

4.12 Sharding and Cross Shard Transaction Requirements

SRS FR 070 The system shall route each incoming key to exactly one of a configured set of independent shard clusters using rendezvous hashing, computing a deterministic numeric score for the key against every configured shard using the SHA two hundred fifty six hash function and selecting the shard with the highest score.

SRS FR 071 The system shall ensure that the routing decision for any given key depends only on the fixed, configured set of shard identifiers and on the key itself, and never on any per process or per run randomised state, so that the identical routing decision is reached by any process constructing the routing table over the same shard set, including the same process across a restart.

SRS FR 072 The system shall reject, before forwarding it to any shard, any ordinary multi key transaction whose keys span more than one shard, returning a clear error to the caller.

SRS FR 073 The system shall provide a dedicated cross shard transaction endpoint supporting atomic, multi key writes and deletes across as many shards as the affected keys are routed to, implemented through a two phase commit coordinator.

SRS FR 074 The system shall, as the first step of any cross shard transaction, durably record which shards that transaction will involve before contacting any of them.

SRS FR 075 The system shall, during the prepare phase of a cross shard transaction, request that every involved shard lock its affected keys using the intent mechanism, aborting the entire transaction if any single shard declines to prepare.

SRS FR 076 The system shall, once every involved shard has successfully agreed to prepare, durably record the final commit decision before instructing any shard to act upon it, this durable recording being the exact point at which the transaction's outcome becomes permanently fixed.

SRS FR 077 The system shall instruct every involved shard to carry out the recorded commit or abort decision, retrying past transient failures until every shard has confirmed completion.

SRS FR 078 The system shall, upon restart after a crash occurring during a cross shard transaction, replay its own durably recorded decision log and resolve any transaction for which no final decision was durably recorded before the crash as an abort.

SRS FR 079 The system shall decline, with a clear explanatory error, any cross shard transaction request received by a gateway instance that was not configured with its own durable data directory.

4.13 Command Line Program Requirements

SRS FR 080 The system shall provide a server program, one running instance of which constitutes exactly one member of a Raft cluster, configurable through command line flags governing network addresses, data directory location, consensus timing, optional Transport Layer Security, and optional operational intelligence settings.

SRS FR 081 The system shall provide a command line client program supporting reading, writing, deleting a key, and checking a node's status, built directly upon the shared client library.

SRS FR 082 The system shall provide a sharding gateway program configured with the network addresses of each independent shard cluster it routes to, which shall function correctly for ordinary routed operations even without a configured data directory, declining only cross shard transaction requests in that configuration.

5 External Interface Requirements

5.1 User Interfaces

SRS IR 001 The system shall provide a command line interface, through the kvctl program, accepting subcommands for get, put, delete, and status operations together with a configurable list of candidate cluster addresses.

SRS IR 002 The system shall provide a graphical, browser rendered dashboard interface, served at the root address and at the dashboard address of any node with the Hypertext Transfer Protocol interface enabled, presenting cluster topology, live performance charts, a key browser, an operational intelligence insight feed, and a natural language query input box, all rendered without any external network resource.

5.2 Hardware Interfaces

The system imposes no requirement for specialised hardware. It requires only a general purpose processor capable of running the Go runtime, an amount of random access memory sufficient for the configured workload, and persistent block storage for each node's data directory, sized according to the configured snapshot threshold and expected key value volume.

5.3 Software Interfaces

SRS IR 003 The system shall depend, for its mandatory functionality, exclusively on the Go standard library, with no third party package required to build, run, or test any part of the system.

SRS IR 004 The system shall, for its strictly optional large language model integration, communicate with an external large language model provider over an encrypted Hypertext Transfer Protocol connection, activated only when a credential is supplied through an environment variable, with the system otherwise fully functional in the complete absence of this integration.

5.4 Communication Interfaces

SRS IR 005 The system shall communicate between cluster members, and between clients and the cluster, using a purpose built binary protocol over Transmission Control Protocol, optionally wrapped in Transport Layer Security.

SRS IR 006 The system shall communicate with human operators, browsers, and external software over standard Hypertext Transfer Protocol, exposing the routes summarised in Table 2.

Table 2: Summary of the Hypertext Transfer Protocol routes exposed by the httpapi component

`Route`	**Purpose**
`GET, PUT, DELETE /api/v1/kv/{key}`	Read, write, or delete one key, forwarded through the leader seeking client library.
`POST /api/v1/txn`	Submit a multi key transaction.
`GET /api/v1/keys?prefix=&limit=`	Browse the locally visible key set of the responding node; explicitly not linearizable.
`GET /api/v1/status`	Retrieve this node's Raft and storage health status, including self reported latency, error rate, queue depth, and log lag.
`GET /api/v1/metrics`	Retrieve metrics in Prometheus text exposition format.

Route	**Purpose**
`GET /api/v1/metrics.json`	Retrieve metrics in plain JSON format.
`GET /api/v1/ai/insights`	Retrieve the operational intelligence insight feed.
`GET /api/v1/ai/recommendation`	Retrieve the current leadership advisor recommendation, if any.
`GET /api/v1/ai/tuning`	Retrieve the current adaptive tuner state.
`GET /api/v1/ai/anomalies`	Retrieve currently and recently detected anomalies.
`GET /api/v1/ai/forecast?series=&horizon=`	Retrieve a least squares trend projection for a named tracked metric series.
`POST /api/v1/ai/query`	Submit a natural language question about cluster state.
`POST /api/v1/ai/transfer`	Request a leadership transfer from this node, which must currently be the leader, to a named target node.
`GET /api/v1/events`	Subscribe to a live Server Sent Events stream of metrics, status, insight, and recommendation updates.
`GET / , GET /dashboard`	Serve the embedded browser dashboard.
`POST /api/v1/xtxn (on cmd/kvgateway)`	Submit a cross shard, multi key atomic write or delete transaction.

6 Non Functional Requirements

6.1 Performance Requirements

SRS NFR 001 The system shall commit a write operation once it has been durably stored on a majority of cluster nodes, without waiting for confirmation from every node, so that the failure or slowness of a strict minority of nodes never blocks a write.

SRS NFR 002 The system shall use a connection per call network protocol chosen deliberately for simplicity and verifiability over maximum raw throughput, on the documented basis that the dominant computational cost in this system lies in the consensus and multi version concurrency control logic rather than in network transport.

SRS NFR 003 The system shall report latency observations with configurable percentile estimates so that an operator can distinguish typical performance from tail latency behaviour.

6.2 Safety and Reliability Requirements

SRS NFR 004 The system shall guarantee that at most one node holds the leader role for any given Raft term across the entire cluster, this property being independently verified through repeated contested election testing.

SRS NFR 005 The system shall guarantee that a minority side of a network partition can never accept or commit a write, preventing the split brain condition in which two disjoint halves of a cluster independently diverge into inconsistent data.

SRS NFR 006 The system shall guarantee that every write reported to a client as successful survives a subsequent crash of the process or the machine on which it was accepted, provided a majority of the cluster remains durable.

SRS NFR 007 The system shall guarantee that the multi version concurrency control garbage collector never removes a version of a key that some currently active transaction still requires in order to observe its consistent snapshot.

6.3 Availability Requirements

SRS NFR 008 The system shall continue to accept and correctly process both reads and writes while a strict minority of cluster nodes are unavailable, whether due to crash, restart, or network partition, provided a majority of the cluster remains reachable and mutually connected.

SRS NFR 009 The system shall automatically recover cluster availability, through a new leader election, within a bounded time after the failure of the current leader, that bound being governed by the configured election timeout range.

6.4 Security Requirements

SRS NFR 010 The system shall support encrypting all network traffic, both internal cluster traffic and external client traffic, using Transport Layer Security, without altering default behaviour for deployments that leave this feature disabled.

SRS NFR 011 The system shall support mutual Transport Layer Security authentication, so that in a suitably configured deployment only clients presenting a certificate signed by a configured trusted authority are permitted to connect at all.

SRS NFR 012 The system shall constrain the optional large language model integration such that it is never permitted to execute an arbitrary, unvalidated instruction of its own choosing, but may only select from a fixed, pre validated, read only operation set that is independently checked before execution.

6.5 Maintainability Requirements

SRS NFR 013 The system shall organise its internal packages in a strict, acyclic dependency hierarchy, such that a change confined to a single low level package, for example the write ahead log, cannot silently alter the observable behaviour of an unrelated higher level package, for example the operational intelligence layer.

SRS NFR 014 The system shall maintain a documented, honest, and current record of known gaps, limitations, and previously discovered and corrected defects, so that the actual state of the system can always be verified against its own documentation.

6.6 Testability Requirements

SRS NFR 015 The system shall provide a dedicated fault injection testing harness capable of simulating network partitions, message delay and loss, and node crash and restart entirely in process, without requiring real, physically separate machines.

SRS NFR 016 The system shall ensure that a simulated node restart within the fault injection harness genuinely reopens and replays that node's actual on disk write ahead log, exercising the identical code path a real process crash and restart would exercise, rather than merely resetting in memory state.

SRS NFR 017 The system shall be verifiable, without race condition, under the standard Go race detector across its full test suite.

SRS NFR 018 The system shall permit the operational intelligence layer to be exercised by automated tests entirely on its own, using simple test doubles for its two required interfaces, with no cluster of any kind running.

6.7 Portability Requirements

SRS NFR 019 The system shall build and run on any operating system and processor architecture combination supported by the standard Go compiler toolchain, without modification.

6.8 Usability Requirements

SRS NFR 020 The system shall present operational intelligence findings, specifically detected anomalies and correlated signals, using transparent, auditable statistical reasoning rather than an opaque, uninterpretable model, on the documented basis that an operator responding to a live incident must be able to understand and trust exactly why a condition was flagged.

SRS NFR 021 The system shall accept plainly phrased natural language questions about cluster state through both the dashboard and the Hypertext Transfer Protocol interface, without requiring the operator to know any query language.

7 System Models

7.1 Raft Node State Model

Every cluster node transitions among exactly three states during normal operation, as illustrated in Figure 2. A node begins as a follower. If a follower's randomised election timer expires without a heartbeat from a recognised leader, it becomes a candidate, increments its term, and requests votes. A candidate that secures a majority of votes for its term becomes leader; a candidate that discovers a higher term, or that receives a valid heartbeat from a legitimate leader, reverts to follower. A leader that discovers a higher term in any message it receives immediately reverts to follower as well, which is

the mechanism by which the cluster always converges on a single leader per term even after a network partition heals.

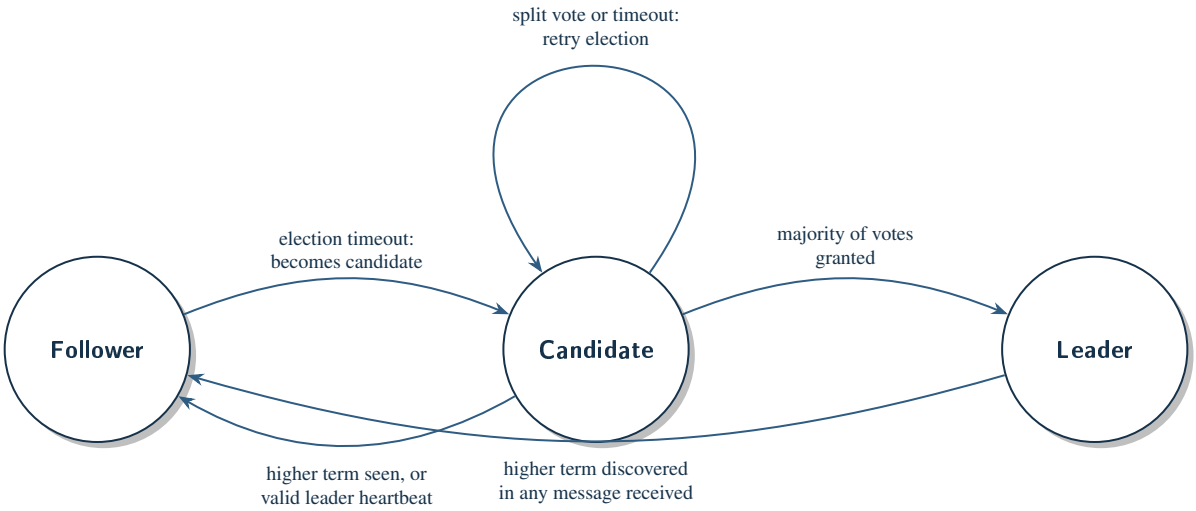


Figure 2: State transition model of a single Raft node.

7.2 Transaction Commit Sequence Model

Figure 3 presents the sequence of events involved in the commit of a single multi key transaction, from the point a client issues a commit request through to the point every replica has deterministically applied the same accept or abort decision. Time runs downward; each vertical dashed line is the lifeline of one participant, and each horizontal arrow is a single message exchanged between two participants at the point in time where it crosses their lifelines.

7.3 Data Flow Overview

At the broadest level, a client request enters the system either through the binary client protocol handled by internal/transport or through the Hypertext Transfer Protocol interface handled by internal/httpapi, which itself uses the internal/client library to reach the current leader. A write request becomes a proposal submitted to internal/raft, is durably logged through internal/wal on a majority of nodes, is applied to internal/mvcc through internal/storage once committed, and is simultaneously measured by internal/metrics, whose resulting observability data is continuously consumed by internal/ai to produce anomalies, correlations, forecasts, and recommendations that flow back out through internal/httpapi to the dashboard and the natural language query interface. A request for a key that is not owned by the current cluster, in a sharded deployment, is instead routed by internal/shard within cmd/kvgateway to the correct independent cluster, with a cross shard write instead flowing through the two phase commit coordinator in cmd/kvgateway before reaching any individual cluster's own internal/raft instance.

8 Data Requirements

8.1 Logical Data Model

The fundamental unit of data in the system is a key, an arbitrary string, mapped to a value, an arbitrary byte sequence. Internally, the multi version concurrency control layer stores not a single value per key but a version chain, each element of which is a tuple consisting of the stored value, the hybrid logical

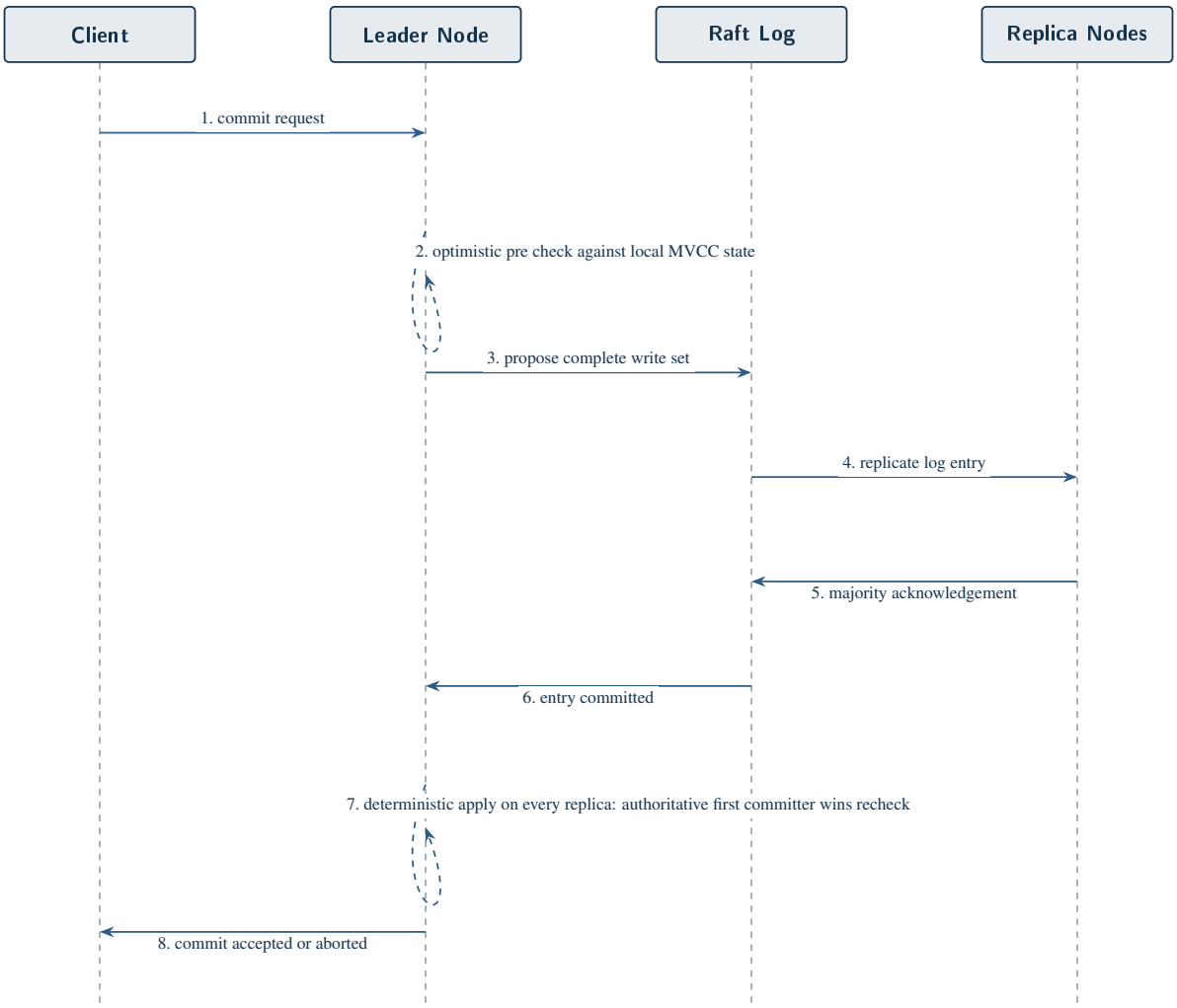


Figure 3: Sequence of events during a transaction commit, showing the optimistic pre check on the leader, the authoritative deterministic apply time conflict check performed identically on every replica, and the resulting accept or abort outcome returned to the client.

clock timestamp at which that version was written, the identifier of the transaction that wrote it, and a boolean flag indicating whether that version represents a deletion. A transaction is itself represented as a beginning timestamp, a set of keys read, and a set of key and value pairs written or marked for deletion, together with the deterministically derived transaction identifier described in Section 4.4.

8.2 Persistent Data

Each cluster node persists, within its own data directory, the Raft replicated log, the current term and vote record, and periodic snapshots, all written through the checksummed, forcibly flushed write ahead log mechanism described in Section 4.2. No other persistent store is used anywhere in the system; the multi version concurrency control version store itself is reconstructed in memory from the persisted Raft log and the most recent snapshot during node startup, consistent with the Raft state machine model in which all durable state ultimately derives from the replicated log.

8.3 Data Integrity Requirements

SRS FR 083 The system shall detect, through a stored per record checksum, any corruption of a write ahead log record caused by a partial write during a crash, and shall treat the log as ending at the first such corrupted or truncated record during replay.

SRS FR 084 The system shall ensure that every replica of the cluster applies the identical sequence of Raft log entries in the identical order, which is the property that guarantees every replica's multi version concurrency control state remains consistent with every other replica's state.

9 Verification, Validation, and Testing Strategy

9.1 Testing Philosophy

The project's own design documentation states an explicit and deliberate preference for testing new functionality against real infrastructure rather than against simplified test doubles wherever that is practical, on the stated basis that a simplified stand in can too easily be constructed in a way that merely confirms an author's existing assumption, silently hiding exactly the kind of subtle timing or interaction defect that only becomes visible when real components genuinely interact. This principle is carried through consistently: the test suite exercises real network sockets, real encrypted Transport Layer Security connections, real multiple node clusters, and, specifically for the sharding tests, genuinely independent real clusters whose data is read back directly afterward to confirm correct placement, rather than merely confirming that a request appeared to succeed when it passed back through the router.

9.2 Fault Injection Testing Harness

The centrepiece of the verification strategy is the internal/fault package, which provides an in process simulated network standing in for real network hardware, and which can be instructed to partition the cluster into groups that cannot exchange any message at all, to introduce artificial delay and message loss at a specified and controllable rate, and to kill and later restart individual simulated nodes, with a restart genuinely reopening and replaying that node's actual on disk write ahead log rather than merely resetting in memory state, so that the identical code path a real process crash and restart would exercise is the one actually exercised by the test.

9.3 Verified Properties

Using this harness, together with the ordinary Go test suite, the following properties have been specifically and directly verified.

- Leader election behaves correctly, and at most one leader can exist for any given term, verified across more than forty repeated fresh runs specifically targeting contested, near simultaneous elections.
- A full network partition never allows the minority side of that partition to accept or commit any write, which is the property that prevents split brain.
- Data survives correctly across a full crash and restart of the entire cluster.
- The cluster continues to make forward progress under continuous write load even while individual nodes are repeatedly and unpredictably crashing and restarting.
- A new voting member can be safely added to a running cluster through the joint consensus mechanism and correctly begins receiving replicated data once that process completes.
- Snapshot isolation and first committer wins conflict detection behave exactly as specified, including that garbage collection never removes a version some still active transaction genuinely still requires.
- The hybrid logical clock preserves causal ordering under injected clock skew.
- The routing table computed by the sharding component agrees across twenty independently constructed instances built over the same shard set, verified by a dedicated regression test, TestRingIsStableAcrossIndependentInstances.
- The full test suite completes without any data race reported by the Go race detector, confirmed across multiple repeated runs.

9.4 Test Suite Organisation

Table 3: Organisation of the fault injection and correctness test suite

File	Coverage
`raft_basic_test.go`	Leader election, log replication, and log compaction with snapshot transfer to a lagging follower.
`fault_partition_test.go`	Full network partitions, asymmetric partitions, and a lossy network under sustained write load, with a full data integrity verification performed afterward.
`fault_crash_test.go`	Whole cluster crash and restart with durability verification, rolling single node crashes under continuous load, and hybrid logical clock causal order preservation under injected skew.

File	Coverage
membership_test.go	Adding a voting member through joint consensus and verifying it receives replicated writes once the configuration finalises.
mvcc_test.go	Snapshot isolation and first committer wins conflict behaviour, disjoint writes never conflicting, and garbage collection never evicting a version an active reader still needs.
leadership_transfer_test.go	Deliberate, voluntary leadership transfer between nodes.
coordinator_test.go (cmd/kvgateway)	Cross shard two phase commit, including a real crash recovery test that closes the coordinator mid transaction and reopens it.
ring_test.go (internal/shard)	Deterministic and stable routing across independently constructed routing rings.

9.5 Acceptance Criteria

A build of the system is considered acceptable for release only when the entire test suite, executed with `go test ./...`, passes cleanly, when the same suite additionally passes cleanly under the Go race detector using `go test ./... -race`, and when both `go build ./...` and `go vet ./...` complete without error across every package.

10 Defect History and Known Limitations

A complete and honest requirements specification must record not only what a system is required to do but also the discipline by which its correctness was actually established, including defects that were found and corrected along the way, and limitations that remain by deliberate, documented choice. This section is included specifically to give an academic evaluator, or any future maintainer, a truthful account of how the system's current guarantees were arrived at, rather than an unqualified assertion that everything works.

10.1 Notable Defects Found and Corrected During Development

- An early version of the fault injection harness reset a simulated node's in memory state on restart rather than genuinely reopening its on disk write ahead log; this was corrected so that a simulated restart now exercises the identical replay code path a real process crash would exercise, which is the property recorded as SRS NFR 016.
- An early implementation of the leadership advisor could recommend a transfer to a follower that had only marginally better health than the current leader, causing the recommendation to oscillate between two nodes on successive polling cycles; a hysteresis margin was introduced so that a transfer is recommended only when a follower is clearly and consistently healthier, as recorded in SRS FR 055.
- The original anomaly detection baseline used a single exponentially weighted moving average,

which allowed a sustained anomaly to gradually retrain the baseline into treating the anomalous condition as normal; a second, slower moving baseline estimate with a variance calculation robust to this drift was introduced, as recorded in SRS FR 050.

- The initial cross shard transaction coordinator did not durably record its commit decision before instructing shards to act upon it, which left a window during which a coordinator crash could leave the transaction's outcome ambiguous; the coordinator was changed to durably record the decision first, as recorded in SRS FR 076, and a dedicated crash recovery test was added to the suite described in Section 9.4.

10.2 Known Limitations

- Cross node clock synchronisation during ordinary production operation relies on the node identifier component of the hybrid logical clock and on each node's physical clock remaining within the configured maximum tolerated drift of its peers, rather than on active, wire level timestamp exchange between nodes, as documented explicitly in SRS FR 006.
- An ordinary, non cross shard transaction is rejected outright if its keys span more than one shard; there is no general purpose distributed transaction capability for arbitrary multi shard reads, only the dedicated two phase commit path described in Section 4.11 for multi shard writes and deletes.
- The operational intelligence layer is intentionally restricted to classical, auditable statistical methods; it contains no trained or learned machine learning model, and the optional large language model integration is constrained at all times to a fixed, pre validated, read only operation set, as recorded in SRS NFR 012.
- Fully automatic leadership transfer is not performed by default; the leadership advisor only ever produces a recommendation, and an actual transfer requires either an explicit operator request or an explicit, separately configured automatic transfer setting, as recorded in SRS FR 056.

11 Appendices

11.1 Appendix A: Requirement Traceability Overview

Every requirement stated in this document carries a unique identifier, as explained in Section 1.2, and is traceable to a specific component described in Section 3.3. Table 4 summarises this mapping at the level of the internal package or command line program primarily responsible for satisfying each numbered range of requirements, so that a reader verifying a particular requirement against the source code knows exactly where to look.

Table 4: Requirement traceability overview, mapping requirement ranges to the primarily responsible component

Requirement Range	Primary Component	Reference
SRS FR 001 - 006	internal/hlc	Section 4.1
SRS FR 007 - 010	internal/wal	Section 4.2
SRS FR 011 - 022	internal/raft	Section 4.3

Requirement Range	Primary Component	Reference
SRS FR 023 - 031	internal/mvcc	Section 4.4
SRS FR 032 - 034	internal/storage	Section 4.5
SRS FR 035 - 039	internal/transport	Section 4.6
SRS FR 040 - 042	internal/client	Section 4.7
SRS FR 043 - 048	internal/metrics	Section 4.8
SRS FR 049 - 062	internal/ai	Section 4.9
SRS FR 063 - 065	internal/tlsutil	Section 4.10
SRS FR 066 - 069	internal/httpapi	Section 4.11
SRS FR 070 - 079	internal/shard, cmd/kvgateway	Section 4.12
SRS FR 080 - 082	cmd/kvnode, cmd/kvctl, cmd/kvgateway	Section 4.13
SRS FR 083 - 084	internal/wal, internal/raft	Section 8.3
SRS IR 001 - 006	cmd/kvctl, internal/httpapi	Section 5
SRS NFR 001 - 021	Cross cutting, verified in Section 9	Section 6

11.2 Appendix B: Command Line Flag Summary

The three command line programs described in Section 3.4 are configured principally through command line flags governing network addresses, the local data directory, Raft consensus timing parameters, and the optional Transport Layer Security and operational intelligence settings described throughout Section 4. Precise flag names, defaults, and descriptions are maintained in each program's own `--help` output and in the project's internal documentation referenced in Section 1.6, which remains the authoritative source for exact flag syntax as the implementation evolves.

11.3 Appendix C: Glossary Cross Reference

The complete glossary of terms, acronyms, and abbreviations used throughout this document is presented in Section 1.5 and is not repeated here; that section should be consulted directly for the definition of any term encountered elsewhere in this specification.